Course Project

May 3, 2026

Abstract

This work presents WarpCrypt, a GPU-accelerated command-line utility for symmetric encryption and decryption, developed to evaluate the performance and design trade-offs of heterogeneous computing. The tool supports AES (128/192/256-bit) and Camellia (128/192/256-bit) in ECB and CTR modes, as well as ChaCha20. A Rust-based front end provides a flexible command-line interface, while performance-critical operations are implemented in CUDA and accessed through a foreign function interface (FFI). WarpCrypt exposes GPU execution parameters, including thread configuration and CUDA streams, enabling exploration of performance trade-offs across workloads. This design allows fine-grained control over parallel execution and supports empirical evaluation of how workload size and configuration impact throughput.

1 Introduction

Modern computing systems demand high-throughput, low-latency cryptographic operations to support applications ranging from secure communications and cloud storage to embedded systems and high-performance computing. Widely used software libraries such as OpenSSL [1] and GnuPG [2] provide robust and highly optimized cryptographic primitives. However, these libraries are optimized primarily for CPU execution, relying on techniques such as vectorization and hardware-specific instruction sets such as AES-NI [3] to achieve performance gains. Although effective, these approaches do not fully leverage the massive parallelism available in modern graphics processing units (GPUs).

Recent work has explored the acceleration of cryptographic workloads using GPUs, demonstrating significant improvements for certain classes of algorithms and modes of operation. In particular, wolfSSL [4] efforts to examine the performance gains of GPU-accelerated encryption using the advanced encryption standard (AES) [5] highlight both the potential and challenges of offloading encryption tasks to CUDA-enabled devices. These efforts show that highly parallel modes, such as electronic codebook (ECB) mode, map well to GPU architectures, while other modes may exhibit limited performance gains due to chained data dependencies.

This project, *WarpCrypt*, builds on these ideas by providing a flexible, GPU-accelerated command-line encryption and decryption utility designed for experimentation and performance analysis. Unlike production-focused libraries, WarpCrypt explicitly exposes GPU execution parameters, including total thread count, thread block size, and number of CUDA streams. This enables users to explore how different configurations impact performance across a variety of cryptographic algorithms and workloads.

WarpCrypt supports multiple symmetric encryption algorithms, including AES and Camellia [6] with 128, 192, and 256-bit keys in both ECB and counter (CTR) modes [7], as well as the ChaCha20 stream cipher [8]. The system is implemented using a hybrid architecture: A Rust-based command-line interface handles user interaction, input/output, and parameter configuration, while performance-critical cryptographic kernels are implemented in C++ using CUDA. These components are connected via a foreign function interface (FFI), allowing the project to combine Rust's safety and ergonomics with the performance and control of CUDA-based GPU programming.

The primary goal of this work is to demonstrate GPU-accelerated cryptography while examining its practical trade-offs involved in such an approach. These include data transfer overhead between host and device memory, kernel launch

configuration, memory access patterns, and the suitability of different cryptographic modes for parallel execution. By making these factors configurable and observable, WarpCrypt serves as both a functional tool and an experimental platform to understand GPU-based cryptographic acceleration.

2 Description of Algorithms

This section provides a brief overview of the cryptographic algorithms and modes of operation supported by WarpCrypt. Specifically, it introduces the Advanced Encryption Standard (AES) and Camellia block ciphers, along with two block cipher modes of operation, namely electronic codebook (ECB) and counter (CTR) modes. Finally, this section describes the ChaCha20 stream cipher, which differs in structure from the block ciphers mentioned above. The focus of these descriptions is the structural properties of each algorithm that are relevant to their implementation and performance on GPU architectures.

2.1 Advanced Encryption Standard

The Advanced Encryption Standard (AES) is a symmetric-key block cipher widely used for the encryption of electronic data [9]. In a symmetric-key system, both the sender and the receiver share the same secret key, which is used for both encryption and decryption. AES operates on fixed-size blocks of data, with a block size of 128 bits (16 bytes). Each 16-byte block is interpreted as a 4×4 column-major matrix of bytes, referred to as the state, which serves as the primary data structure throughout the algorithm.

Encryption in AES proceeds through a series of rounds, each consisting of well-defined transformations applied to the state. The number of rounds depends on the key size; for AES-128, which uses a 128-bit key, a total of 10 rounds are performed. For AES-192, 12 rounds are performed and for AES-256, 14 rounds are performed. Before the rounds, a key expansion step derives a set of round keys from the original secret key, producing 11 distinct 128-bit keys (one for the initial step and one for each round).

Each round applies transformations that provide confusion and diffusion.

- `SubBytes`: A non-linear substitution step where each byte is replaced using a fixed lookup table (S-box).
- `ShiftRows`: A transposition step in which each row of the state is cyclically shifted by a different offset.
- `MixColumns`: A linear transformation applied to each column of the state using finite field arithmetic.
- `AddRoundKey`: Each byte of the state is combined with a corresponding byte of the round key using the XOR operation.

The first round begins with an `AddRoundKey` operation. This is followed by nine identical rounds consisting of `SubBytes`, `ShiftRows`, `MixColumns`, and `AddRoundKey`. The final round omits the `MixColumns` step and consists only of `SubBytes`, `ShiftRows`, and `AddRoundKey`. After the final round, the transformed state represents the encrypted output block.

Decryption in AES follows a structure similar to encryption, but applies the inverse of each transformation in reverse order. Using the same expanded round keys (applied in reverse), the algorithm performs `AddRoundKey`, followed by inverse operations: `InvShiftRows`, `InvSubBytes`, `InvMixColumns`. The process begins with an initial `AddRoundKey` using the final round key, followed by a number of rounds (corresponding to the key size) of inverse transformation. These operations effectively undo the confusion and diffusion introduced during encryption, recovering the original plaintext block from the ciphertext.

2.2 Camellia

Camellia is a symmetric-key block cipher with security and performance comparable to AES, and is standardized for use in various cryptographic applications. Like AES, Camellia operates on 128-bit (16-byte) blocks of data and supports key sizes of 128, 192, and 256 bits. However, unlike the structure of the AES substitution–permutation network, Camellia is based on a Feistel network, which divides each input block into two halves and processes them iteratively.

In Camellia, the 128-bit input block is divided into left and right 64-bit halves. Encryption proceeds through a series of Feistel rounds, where in each round one half is transformed using a round function and combined with the other half using XOR, after which the halves are swapped. The round function itself includes substitution using S-boxes, permutation operations, and key mixing. Depending on the key size, Camellia uses either 18 rounds (for 128-bit keys) or 24 rounds (for 192- and 256-bit keys).

In addition to the standard Feistel rounds, Camellia incorporates additional layers known as `FL` and `FLINV` functions at specific points in the algorithm. These functions introduce non-linear and linear transformations that enhance security and contribute to resistance against known cryptographic attacks. As with AES, a key schedule expands the original secret key into a set of round keys used throughout the encryption process.

Camellia achieves confusion and diffusion through its iterative Feistel structure, with each round progressively transforming the data until the final ciphertext is produced.

One advantage of Feistel-based ciphers such as Camellia is that encryption and decryption share nearly identical structures. Decryption is performed by applying the same sequence of operations as encryption but with the round keys used in reverse order. Because each Feistel round is inherently reversible, there is no need to define separate inverse functions for each transformation, unlike in AES. This structural symmetry simplifies implementation while still ensuring that the original plaintext can be correctly recovered from the ciphertext.

2.3 Block Cipher Modes of Operation

Block ciphers such as AES and Camellia operate on fixed-size blocks of data, but real-world inputs typically span many blocks. Certain *modes of operation* [7] define how these individual block encryptions are composed to securely process arbitrary-length data. In this work, two modes are considered: Electronic Codebook (ECB) and Counter (CTR). These modes illustrate both structural and security differences, as well as their suitability for parallel execution.

2.3.1 Electronic Codebook (ECB)

Electronic codebook (ECB) is the simplest mode of operation. In ECB mode, each plaintext block is encrypted independently of all other blocks, with no shared state between them. Mathematically, this is expressed as $C_i = E_K(P_i)$, where P_i is the ith block of plaintext, C_i is the corresponding ciphertext block and $E_K(\cdot)$ represents the block cipher encryption function under key K. The complete encryption and decryption processes for ECB mode are illustrated in Figure 1.

Because ECB applies the same deterministic transformation to each block, identical plaintext blocks will always produce identical ciphertext blocks. While this makes ECB simple and highly parallelizable, it also introduces significant security weaknesses.

To illustrate this, consider the plaintext image shown in Figure 2. This image contains large regions of identical or similar pixel values. When encrypted using ECB mode, the resulting ciphertext image (also shown in Figure 2) still reveals the underlying structure of the original image. Distinct visual features remain recognizable because identical input blocks map to identical output blocks. For this reason, ECB mode is generally considered insecure for most practical applications.

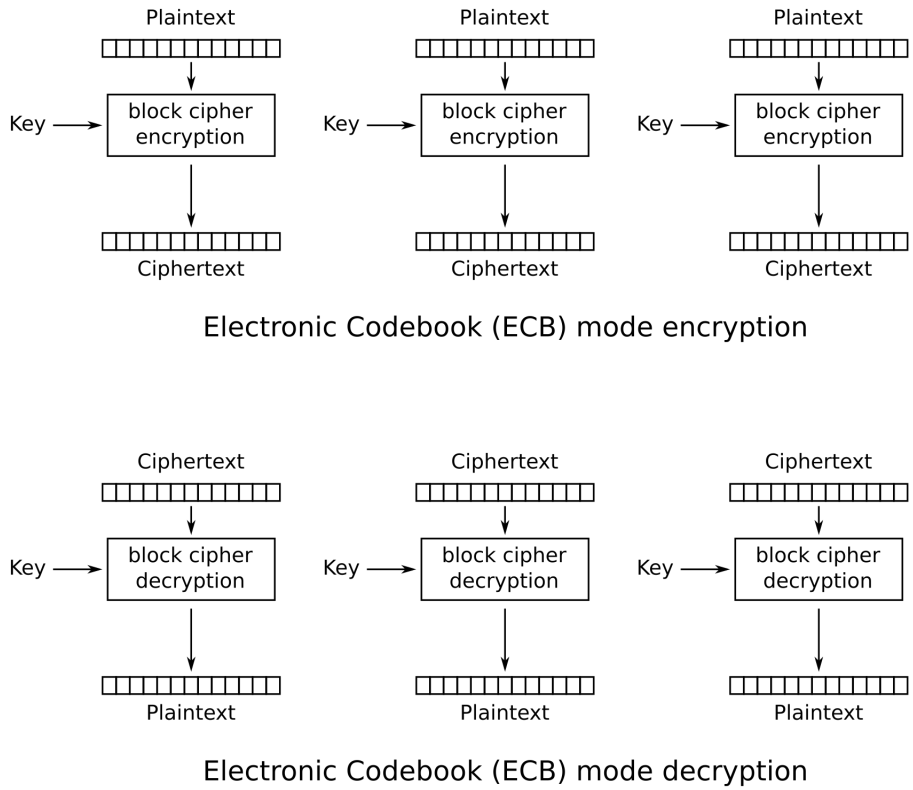


Figure 1: ECB mode encryption and decryption. *Wikimedia Commons, CC BY-SA 2.0.*

Despite these weaknesses, ECB is useful in this project as a baseline for performance evaluation. Its lack of inter-block dependencies makes it trivially parallelizable, as each block can be processed independently by a separate GPU thread without synchronization.

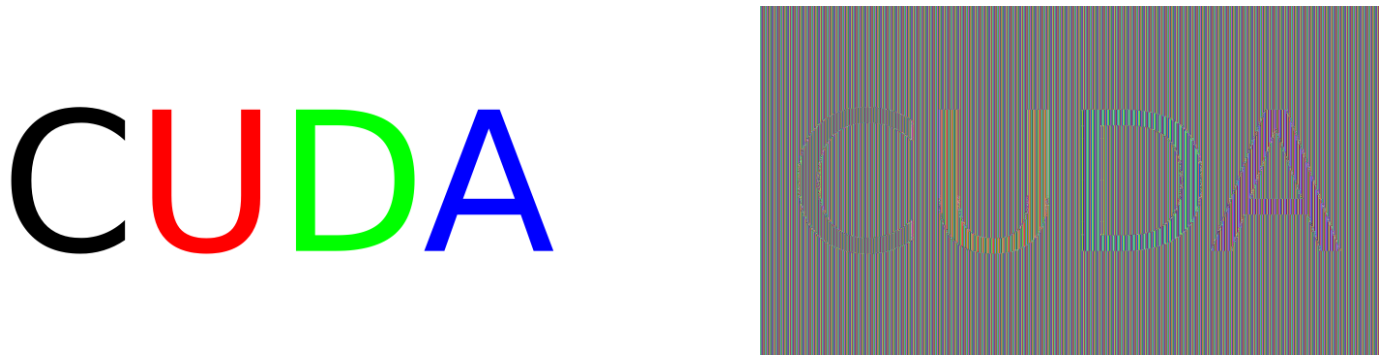
2.3.2 Counter Mode (CTR)

Counter mode (CTR) addresses the primary weakness of ECB by introducing variability between blocks. Instead of directly encrypting the plaintext, CTR mode encrypts a sequence of unique input values and combines the result with the plaintext using the XOR operation. This can be expressed as $C_i = P_i \oplus E_K(n \| g(i))$, where n is a nonce (a value that must be unique per encryption), $\|$ denotes concatenation, $g(i)$ is a function that produces a unique counter value for each block (commonly $g(i) = i$), and $\oplus$ denotes XOR. The overall encryption and decryption processes for CTR mode are shown in Figure 3.

Because each block uses a different counter value, even identical plaintext blocks produce different ciphertext blocks (assuming the counter or nonce is different). This eliminates the pattern leakage seen in ECB mode. Returning to the example in Figure 2, when the same input image is encrypted using CTR mode, the output appears pseudorandom and no visible structure of the original image remains, as shown in Figure 4.

CTR mode also has the property that encryption and decryption are identical operations since XOR is its own inverse. This simplifies the implementation compared to modes that require different decryption procedures.

From a performance perspective, CTR retains the parallelizability of ECB while improving security. Each counter value can be generated independently, allowing multiple blocks to be processed concurrently without dependencies. This makes



(a) Plaintext image. (b) The same image encrypted using AES-ECB.

Figure 2: Encryption using ECB showing that identical plaintext blocks produce identical ciphertext blocks.

CTR particularly well-suited for GPU execution, where thousands of threads can independently generate keystream blocks and apply XOR operations in parallel.

2.4 ChaCha20

ChaCha20 is a high-performance symmetric stream cipher used heavily in networking applications. Unlike block ciphers such as AES and Camellia, which operate on fixed-size blocks of plaintext, ChaCha20 generates a continuous stream of pseudorandom data (a keystream) that is combined with the plaintext using the XOR operation.

ChaCha20 operates on a 512-bit (64-byte) internal state, which is initialized using a 256-bit secret key, a 32-bit block counter, and a 96-bit nonce. The state is typically represented as a 4×4 matrix of 32-bit words. Encryption proceeds by applying a sequence of rounds to this state, producing a 512-bit output block that serves as the keystream. This keystream is then XORed with a 64-byte block of plaintext to produce the corresponding ciphertext block: $C_i = P_i \oplus \texttt{ChaCha20}(K, n, g(i))$.

The core of the ChaCha20 algorithm consists of a repeated operation known as the quarter round, which mixes four 32-bit words using a combination of addition, XOR, and bitwise rotation. These quarter rounds are applied in a structured sequence across the state matrix, alternating between column rounds and diagonal rounds to ensure thorough diffusion. ChaCha20 performs 20 such rounds (organized as 10 double-rounds), after which the transformed state is added to the original state to produce the final output block.

Decryption in ChaCha20 is identical to encryption. Because the keystream is generated independently of the plaintext, applying the XOR operation with the same keystream to the ciphertext recovers the original plaintext. This symmetry simplifies implementation and avoids the need for separate inverse operations.

From a parallelization perspective, ChaCha20 is highly efficient. Each keystream block is generated independently based on the key, nonce, and counter value, allowing multiple blocks to be computed in parallel without any interdependence. This makes the algorithm particularly well-suited for GPU execution, where many threads can simultaneously generate keystream blocks and apply XOR operations.

Conceptually, the structure of ChaCha20 is very similar to CTR mode used with block ciphers. In both cases, a sequence of unique input values (counters) is transformed into a pseudorandom keystream, which is then XORed with the plaintext. The key difference is that ChaCha20 is designed as a native stream cipher, whereas CTR mode adapts a block cipher to achieve similar behavior.

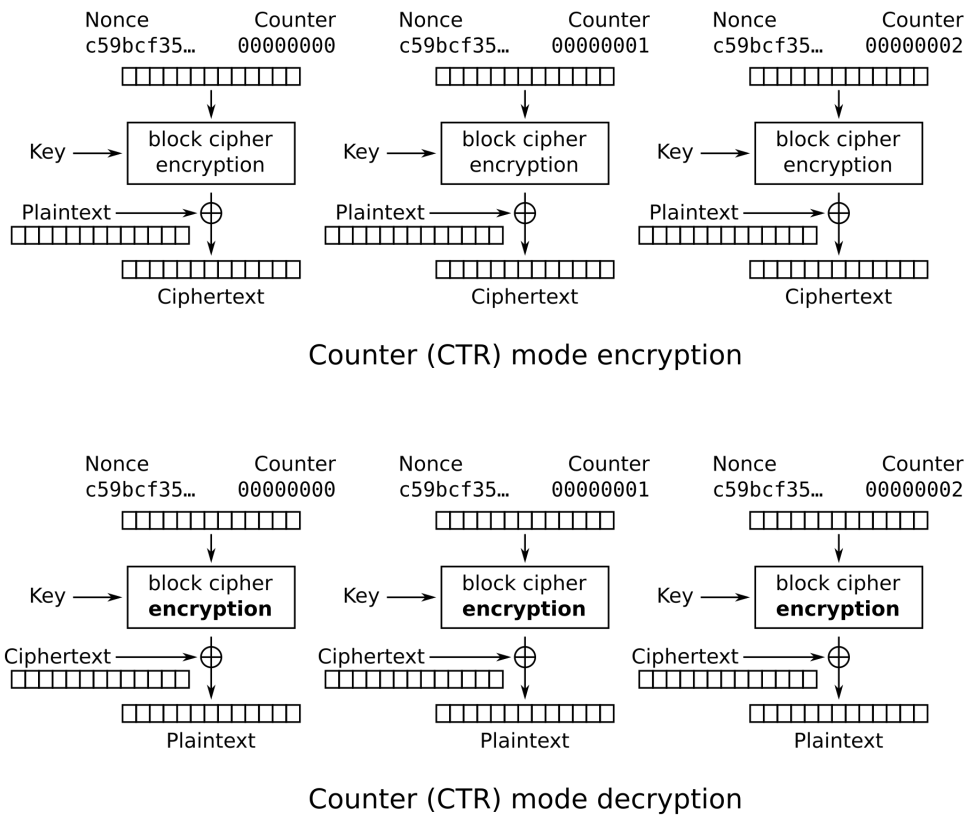


Figure 3: CTR mode encryption and decryption. *Wikimedia Commons, CC BY-SA 2.0.*

3 Implementation and Design

WarpCrypt is a hybrid system that combines a high-level command-line interface with a GPU-accelerated cryptographic backend. The system separates concerns between host-side orchestration and device-side computation, allowing for both ease of use and fine-grained control over execution. In particular, the design emphasizes the exposure of GPU configuration parameters to the user, allowing experimentation with different execution strategies while maintaining a consistent interface for cryptographic operations.

3.1 Project Design

The overall architecture of WarpCrypt follows a layered design in which a Rust-based front end manages input/output, parameter parsing, and preprocessing, while a C++/CUDA backend performs the core cryptographic computations. Execution begins in Rust, where command-line arguments are parsed into a structured configuration describing the requested operation, including the algorithm, key size, mode, and GPU parameters.

Input data is read from disk into a contiguous byte buffer, after which preprocessing steps are applied as needed. For block cipher modes such as AES-ECB and Camellia-ECB, the implementation enforces block alignment requirements. During encryption, optional PKCS#7 padding [10] is applied to ensure that the input length is a multiple of the 16-byte block size, while during decryption, alignment is validated prior to processing. These steps are performed entirely on the host side to simplify the device implementation.

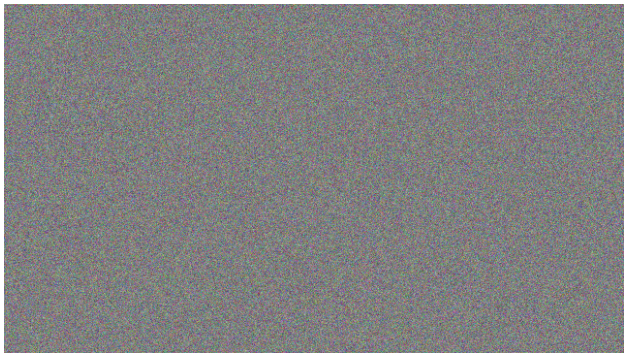


Figure 4: AES-CTR encryption of the image shown in Figure 2a

After preprocessing, the input buffer, along with the key, initialization vector (IV, which encapsulates both the nonce and initial counter), and a fully specified request structure, are passed to the GPU backend via a foreign function interface (FFI). This interface, implemented using the Rust `cxx` crate, exposes a single entry point, `execute_crypto`, which encapsulates all parameters required for execution. The request structure includes not only the selected algorithm and operation, but also GPU-specific parameters such as the number of thread blocks, threads per block, and CUDA streams.

On the device side, WarpCrypt maps independent units of work to individual GPU threads. For block ciphers, each thread processes a single 16-byte block, while for ChaCha20 each thread generates and applies a 64-byte keystream block. This design allows for a high degree of parallelism, as threads operate independently with no inter-block dependencies in ECB, CTR, or ChaCha20 modes. Input data is transferred to device global memory prior to kernel execution, and results are copied back to host memory upon completion. CUDA streams can be used to enable concurrent execution and overlap between computation and data transfer, depending on the configuration.

After GPU execution, the Rust layer performs any necessary postprocessing, such as removing PKCS#7 padding during decryption, before writing the final output to disk. This division of responsibilities keeps the GPU kernels focused on computation, while maintaining flexibility and safety in the host-side implementation.

3.2 Command-Line Functionality

WarpCrypt provides a command-line interface for configuring both cryptographic parameters and GPU execution settings. The interface is structured around two primary operations, `encrypt` and `decrypt`, each of which accepts a set of options specifying the algorithm, key material, input/output files, and execution parameters.

Users can select from multiple algorithms, including AES and Camellia in ECB and CTR modes, as well as the ChaCha20 stream cipher. Key material may be provided either directly as a hexadecimal string or via an external file, and initialization vectors or nonces are supplied where required (e.g., for CTR mode and ChaCha20). Input and output are handled as binary files, and the entire file is processed as a contiguous byte array.

In addition to cryptographic options, the interface exposes GPU configuration parameters that directly influence execution. These include the number of CUDA thread blocks (`--num_blocks`), the number of threads per block (`--block_size`), and the number of CUDA streams (`--num_streams`). By exposing these parameters, WarpCrypt enables users to explore how different configurations affect performance across algorithms and workloads.

For example, the following command encrypts a file using ChaCha20 with a specified thread block size and multiple CUDA streams:

```
warpcrypt encrypt \
    --algorithm chacha20 \
    --key-hex 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f \
    --iv 010000000000000000000004a00000000 \
    --input plaintext.bin \
    --output ciphertext.bin \
    --block-size 192 \
    --num-streams 2
```

Similarly, the following command decrypts a file using AES in ECB mode with a key read from a file and a specified number of CUDA thread blocks:

```
warpcrypt decrypt \
    --algorithm aes-ecb \
    --key-file key.bin \
    --input aes_ciphertext.bin \
    --output aes_plaintext.bin \
    --num-blocks 1024
```

This design emphasizes transparency and control. While many cryptographic tools abstract away hardware-specific details, WarpCrypt exposes them explicitly, allowing users to treat the GPU as a tunable resource. This makes the system not only a functional encryption utility, but also a platform for experimenting with and understanding GPU-accelerated computation.

4 Benchmarks

This section evaluates WarpCrypt's performance across all currently-implemented cryptographic algorithms, modes of operation, and input sizes. Benchmarks were conducted using Rust's built-in benchmark framework, measuring end-to-end execution time, including host-device memory transfers and kernel execution. GPU benchmarks were performed using varying numbers of CUDA streams (1, 2, 4, and 8) to assess the impact of concurrency, while CPU baselines were obtained using RustCrypto [11] implementations of the same algorithms. The tests were conducted on input sizes of 1 MB, 16 MB, 256 MB, and 1 GB, capturing both small and large workload behavior. The results are grouped by algorithm and mode of operation, with key sizes (128, 192, and 256 bits) presented within each figure to highlight their impact on performance.

All benchmarks were performed on a consumer desktop with an AMD Ryzen 7 9700X Processor and a GeForce RTX 5070 GPU with 12GB of GDDR7 RAM.

4.1 AES

Figures 5 and 6 show the performance of AES in ECB and CTR modes, respectively, across varying input sizes and CUDA stream configurations. Across all key sizes, the CPU implementations perform strongly, likely due to hardware acceleration and compiler optimizations. For smaller inputs (1 MB and 16 MB), CPU execution consistently outperforms GPU execution, as kernel launch overhead and host-device memory transfer dominate total execution time.

As the input size increases, GPU performance becomes more competitive. At 256 MB and above, GPU implementations begin to outperform the CPU, with a higher number of CUDA streams providing additional performance gains. This trend reflects an improved amortization of overhead and an increased opportunity for overlapping computation and data transfer.

Comparing modes, AES-CTR on the CPU achieves slightly lower throughput than AES-ECB, at approximately 80% of ECB performance. This difference is expected due to the additional counter generation and XOR operations required in CTR mode. However, on the GPU, the performance of ECB and CTR is nearly identical, suggesting that these additional operations are effectively hidden by parallel execution.

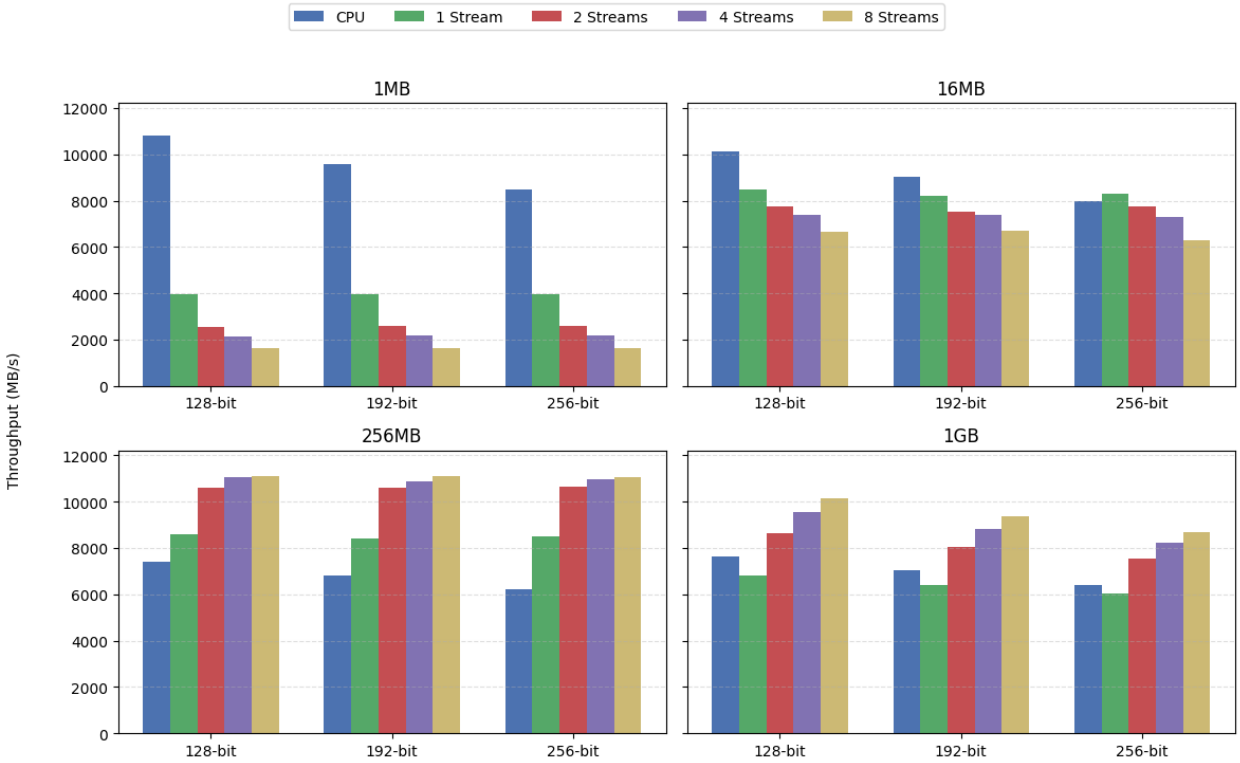


Figure 5: Throughput of AES-ECB implementations across workloads and key size.

4.2 Camellia

Figures 7 and 8 show the performance of Camellia in ECB and CTR modes. In contrast to AES, the CPU implementations of Camellia perform significantly worse and exhibit relatively constant throughput across all input sizes. This behavior suggests the absence of hardware acceleration and limited optimization compared to AES. The CPU implementations of Camellia exhibit significantly lower throughput, to the extent that they are not clearly visible at the scale of the plotted results.

However, GPU implementations perform similarly to those of AES, demonstrating strong scalability with increasing input size. The effect of CUDA streams follows a similar pattern to AES: for smaller inputs, increasing the number of streams provides little benefit and can even degrade performance due to overhead. At larger input sizes, additional streams improve throughput by enabling greater concurrency.

No significant performance difference is observed between ECB and CTR modes for Camellia on either the CPU or the GPU. This aligns with expectations, as both modes allow independent block processing and introduce minimal additional computation relative to the overall workload.

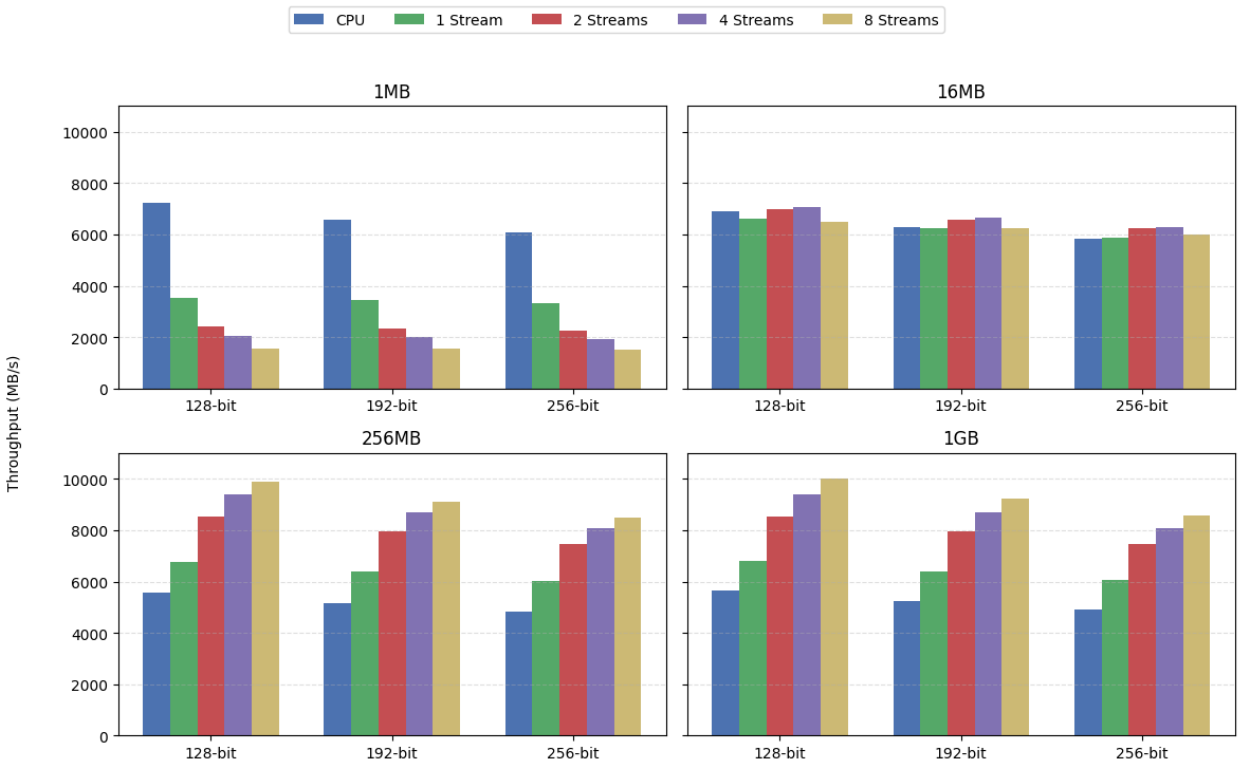


Figure 6: Throughput of AES-CTR implementations across workloads and key size.

4.3 ChaCha20

Figure 9 shows the performance of the ChaCha20 stream cipher across varying input sizes and CUDA stream configurations. The CPU implementation exhibits relatively constant throughput across all input sizes, indicating that performance is dominated by computation rather than overhead.

For small inputs (1 MB), the GPU implementation with a single stream provides the best performance among GPU configurations, while higher stream counts introduce unnecessary overhead. At 16 MB, all GPU configurations begin to outperform the CPU, indicating that the workload is large enough to amortize GPU overhead.

As with AES and Camellia, the benefits of increased CUDA streams become more apparent at larger input sizes. At 256 MB and above, configurations with more streams achieve higher throughput, reflecting improved utilization of the GPU and better overlap of computation and memory operations.

Overall, ChaCha20 demonstrates strong performance on the GPU due to its stream cipher structure, with scalability characteristics similar to those observed for CTR mode in block ciphers.

5 Conclusion

This work presented WarpCrypt, a GPU-accelerated command-line utility for symmetric encryption supporting AES, Camellia, and ChaCha20. By combining a Rust-based front end with a CUDA backend, the system provides a flexible interface and high-performance computation while exposing GPU execution parameters for experimentation.

Results show that GPU acceleration significantly improves performance for large workloads, particularly for algorithms

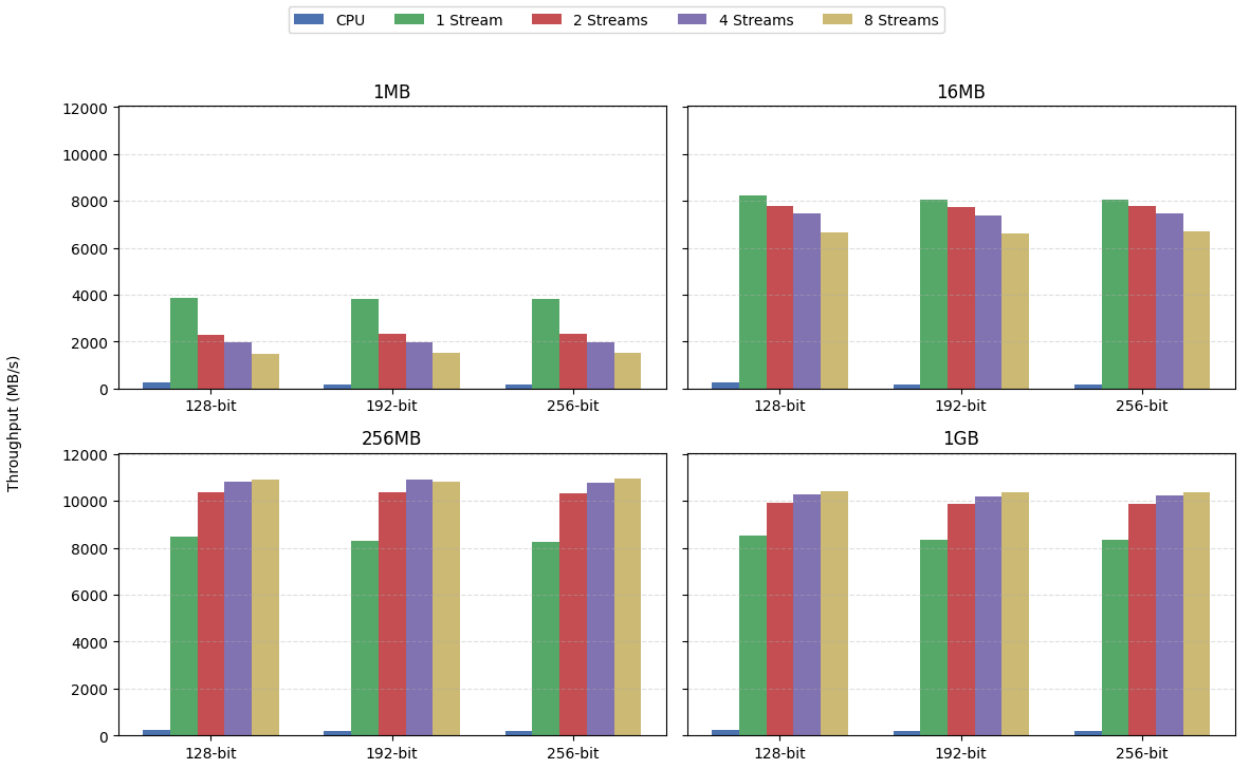


Figure 7: Throughput of Camellia-ECB implementations across workloads and key size.

without hardware acceleration or with strong data parallelism. However, for smaller inputs, CPU implementations remain more efficient due to lower overhead. These results highlight that GPU acceleration is workload- and algorithm-dependent.

Future work includes adding authenticated encryption modes such as AES-GCM [12] and storage-focused modes such as XTS-AES [13], as well as further optimization of memory management, kernel design, and multi-GPU execution.

References

[1] The OpenSSL Project, *OpenSSL: The Open Source Toolkit for SSL/TLS*, https://www.openssl.org/

[2] The GnuPG Project, *The GNU Privacy Guard*, https://www.gnupg.org/

[3] Kahraman Akdemir et al. (2012), *Breakthrough AES Performance with Intel AES New Instructions*, https://www.intel.com/content/dam/develop/external/us/en/documents/10tb24-breakthrough-aes-performance-with-intel-aes-new-instructions-final-secure-165683.pdf

[4] wolfSSL Inc.,*wolfSSL Embedded SSL/TLS Library*, https://www.wolfssl.com/

[5] wolfSSL Inc. (2024), *Accelerating AES Encryption with Nvidia CUDA: WolfCrypt Performance Boost*, https://www.wolfssl.com/accelerating-aes-encryption-with-nvidia-cuda-wolfcrypt-performance-boost/

[6] Mitsuru Matsui, Shiho Moriai, and Junko Nakajima (2004), *A Description of the Camellia Encryption Algorithm*, RFC 3713, https://www.rfc-editor.org/info/rfc3713

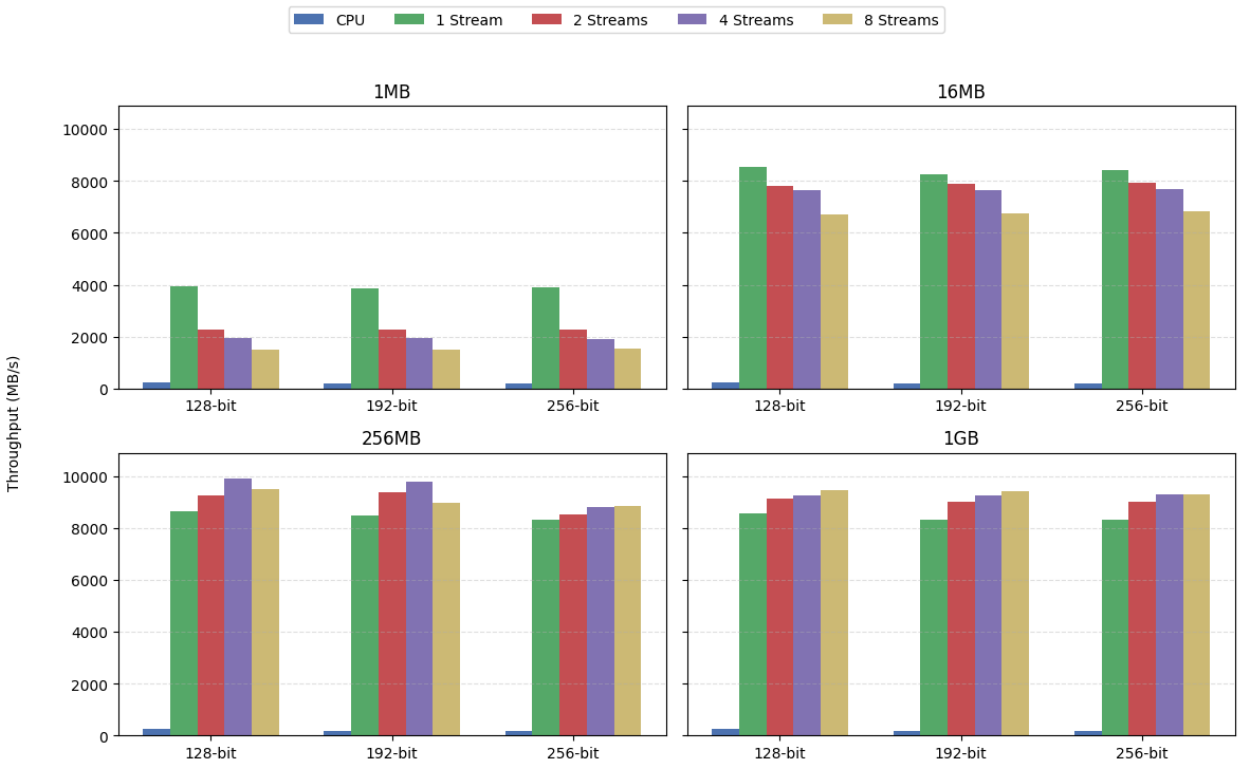


Figure 8: Throughput of Camellia-CTR implementations across workloads and key size.

[7] Morris Dworkin (2001), *Recommendation for Block Cipher Modes of Operation: Methods and Techniques*, NIST Special Publication 800-38A.

[8] Yoav Nir and Adam Langley (2018), *ChaCha20 and Poly1305 for IETF Protocols*, RFC 8439, https://www.rfc-editor.org/info/rfc8439

[9] National Institute of Standards and Technology (2001), *Advanced Encryption Standard (AES)*, FIPS 197, https://doi.org/10.6028/NIST.FIPS.197-upd1

[10] R. Housley (2009), *Cryptographic Message Syntax (CMS)*, RFC 5652, https://www.rfc-editor.org/info/rfc5652

[11] The RustCrypto Project, *RustCrypto*, https://github.com/rustcrypto

[12] Morris Dworkin (2007), *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, NIST Special Publication 800-38D.

[13] Morris Dworkin (2010), *Recommendation for Block Cipher Modes of Operation: the XTS-AES Mode for Confidentiality on Storage Devices*, NIST Special Publication 800-38E.

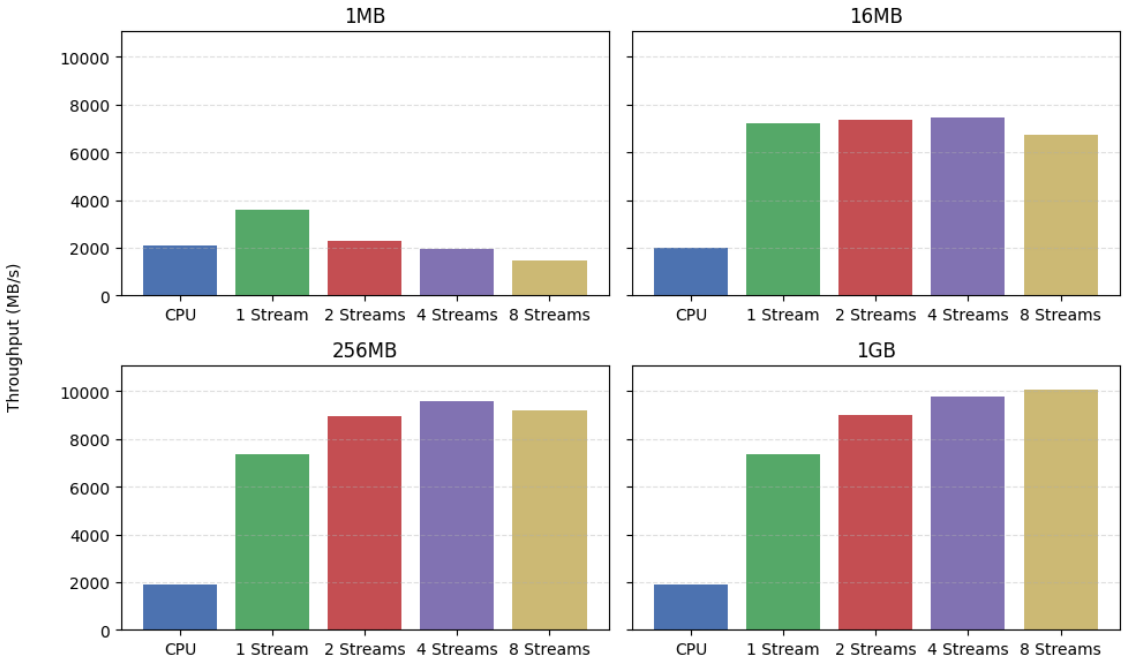


Figure 9: Throughput of ChaCha20 implementations across workloads and execution configurations.